application among multiple machines. Also, decoupling the parts of your architecture such that one component does not rely on the uptime of the others is important. For example, the web server could deliver cached content if the database is not reachable.

The services provided by AWS offer different types of *failure resilience*. Resilience is the ability to deal with a failure with no or little effect on the user. You will learn about the resilience guarantees of major services in this chapter. But, in general, if you are unsure about the resilience capabilities of an AWS service, refer to the Resilience section of the official documentation for that service. A fault-tolerant system is very resilient to failure. We group AWS services into the following three categories:

- *No guarantees (single point of failure)*—No requests are served if failure occurs.
- *High availability*—In the case of failure, recovery can take some time. Requests might be interrupted.
- *Fault tolerant*—In the case of failure, requests are served as before without any availability problems.

The most convenient way to make your system fault tolerant is to build the architecture using only fault-tolerant services, which you will learn about in this chapter. If all your building blocks are fault tolerant, the whole system will be fault tolerant as well. Luckily, many AWS services are fault tolerant by default. If possible, use them. Otherwise, you'll need to deal with the consequences and handle failures yourself.

Unfortunately, one important service isn't fault tolerant by default: EC2 instances. Virtual machines aren't fault tolerant. This means an